

Введение в WPF и XAML

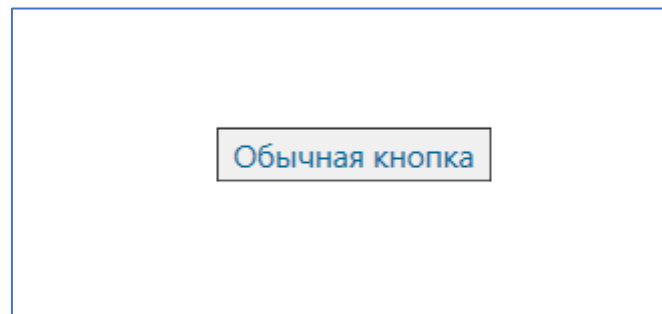
XAML – это язык разметки на основе XML, используемый для создания и инициализации объектов в WPF.

Основные характеристики XAML:

- Является диалектом XML
- Иерархическая структура с одной корневой вершиной
- Названия элементов чувствительны к регистру и совпадают с именами классов .NET
- Поддерживает два стиля задания атрибутов:
- Встроенный синтаксис: `<Button Background="Blue" />`
- Синтаксис свойств: с использованием дочерних элементов для сложных значений

Пример встроенного синтаксиса XAML:

```
9      <Grid>
10
11      <Button x:Name="Btn1"
12              HorizontalAlignment="Center"
13              VerticalAlignment="Center"
14              Width="150"
15              Height="30"
16              FontSize="17"
17              Content="Обычная кнопка"
18              Foreground=■ "#006699"
19              Background=■ "#f0f0f0"
20              BorderBrush=□ "#303030" />
21      </Grid>
</Window>
```

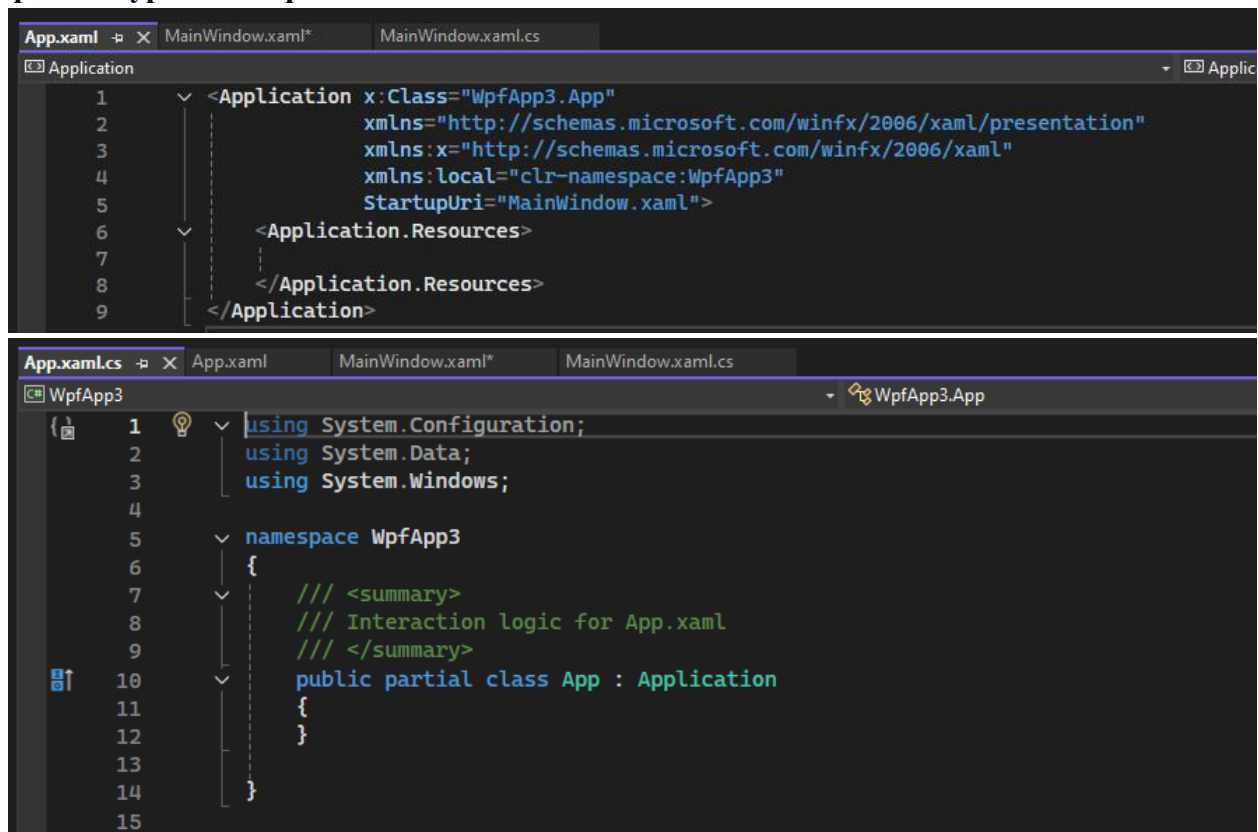


Категории элементов XAML

XAML использует четыре основные категории элементов:

Категория	Описание	Пример
Панели размещения	Контейнеры для расположения дочерних элементов	Canvas, Grid, StackPanel
Элементы управления	Интерактивные компоненты интерфейса	Button, TextBox, CheckBox
Элементы документов	Элементы для работы с текстом и документами	TextBlock, Paragraph, RichTextBox
Графические фигуры	Примитивы для рисования	Rectangle, Ellipse, Path

Архитектура WPF приложения



Объект Application

`Application` – это ключевой объект, который управляет временем жизни всего приложения. Он отвечает за:

- Подключение к модели событий операционной системы
- Управление видимыми окнами
- Освобождение ресурсов
- Контроль глобального состояния приложения

Жизненный цикл `Application`:

- Конструируется объект `Application`
- Вызывается метод `Run()`
- Выполняется событие `Application.Startup`
- Конструируются объекты `Window` (окна приложения)
- Приложение выполняет работу
- Вызывается метод `Application.Shutdown()`
- Вызывается метод `Application.Exit()`

Доступ к `Application`:

// Получить доступ к текущему объекту приложения

```
Application currentApp = Application.Current;
```

В проекте автоматически создаются файлы:

`App.xaml` – определение ресурсов и параметров приложения

`App.xaml.cs` – код-бихайнд для инициализации приложения

Объект Window

Window – это корневой контейнер для пользовательского интерфейса приложения.

- Является корневым элементом в XAML-файле
- Содержит один диспетчер компоновки, который в свою очередь содержит остальные элементы
- Представляет отдельное окно в приложении

Разделение дизайна и кода

WPF спроектирована в рамках концепции четкого отделения:

Дизайнерская часть (XAML): определяет структуру и внешний вид интерфейса

Кодовая часть (C#): реализует функциональность и обработку событий

Каждый XAML-файл имеет соответствующий файл code-behind (кодовой части):

MainWindow.xaml – дизайн

MainWindow.xaml.cs – логика

Панели размещения (Layout Panels)

Концепция панелей размещения

Панель размещения (диспетчер компоновки) – это контейнерный элемент управления, который определяет, как расположены дочерние элементы внутри него.

Все панели наследуются от абстрактного класса `System.Windows.Controls.Panel`.

Canvas – абсолютное позиционирование

Определение:

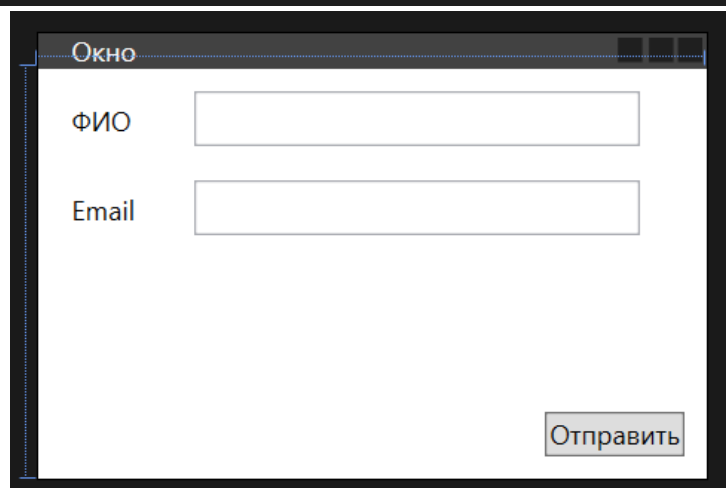
Canvas – панель, которая поддерживает абсолютное позиционирование содержимого. Элементы остаются в точности там, где были размещены во время проектирования.

Характеристики:

- Использует абсолютные координаты (Left, Top, Right, Bottom)
- При уменьшении окна элементы могут выходить за границы и становиться невидимыми
- Не изменяются динамически при изменении размеров окна

Пример использования Canvas:

```
8      Title="Окно" Height="200" Width="300">
9      <Grid>
10     <Canvas>
11         <Label Canvas.Left="10" Canvas.Top="10" Content="ФИО" />
12         <TextBox Canvas.Left="70" Canvas.Top="10" Width="200" Height="25" />
13         <Label Canvas.Left="10" Canvas.Top="50" Content="Email" />
14         <TextBox Canvas.Left="70" Canvas.Top="50" Width="200" Height="25" />
15         <Button Canvas.Right="10" Canvas.Bottom="10" Content="Отправить" />
16     </Canvas>
17 </Grid>
18 </Window>
```



WrapPanel – динамическое переносимое содержимое

Определение:

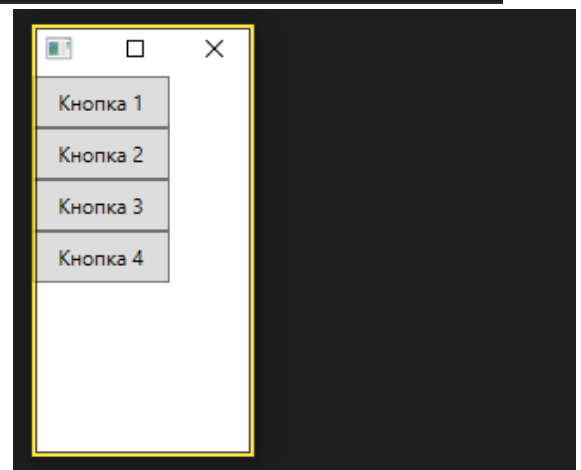
WrapPanel позиционирует содержимое слева направо (или сверху вниз), переносит на следующую строку (столбец) по достижении границы панели.

Характеристики:

- Выводит элементы последовательно в одном направлении
- При достижении границы окна автоматически переносит элементы на новую строку/столбец
- При изменении размеров окна перераспределяет элементы динамически
- Свойство Orientation определяет направление: Horizontal (по умолчанию) или Vertical

Пример использования WrapPanel:

```
9      <Grid>
10
11      <WrapPanel Orientation="Horizontal">
12          <Button Width="80" Height="30" Content="Кнопка 1" />
13          <Button Width="80" Height="30" Content="Кнопка 2" />
14          <Button Width="80" Height="30" Content="Кнопка 3" />
15          <Button Width="80" Height="30" Content="Кнопка 4" />
16      </WrapPanel>
      </Grid>
```



При уменьшении ширины окна кнопки автоматически перенесутся на следующие строки.

StackPanel – линейное размещение

Определение:

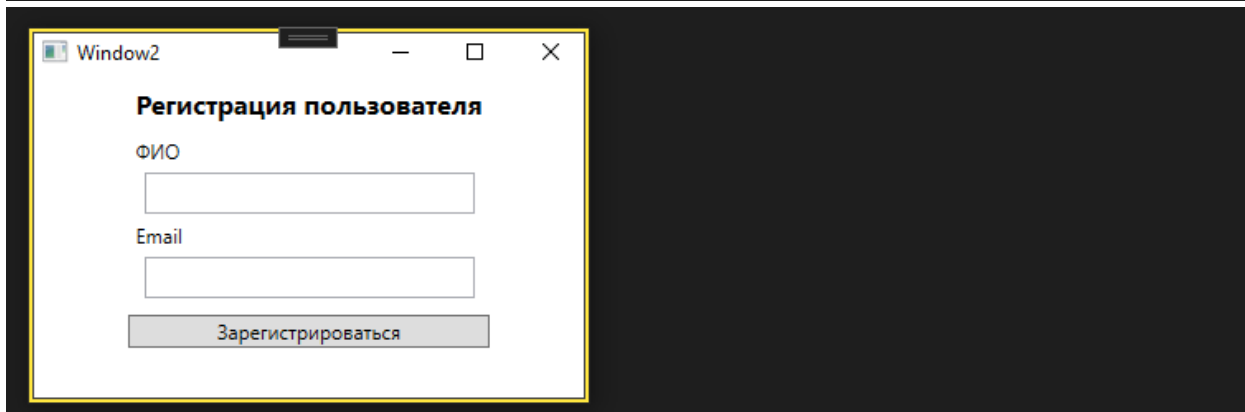
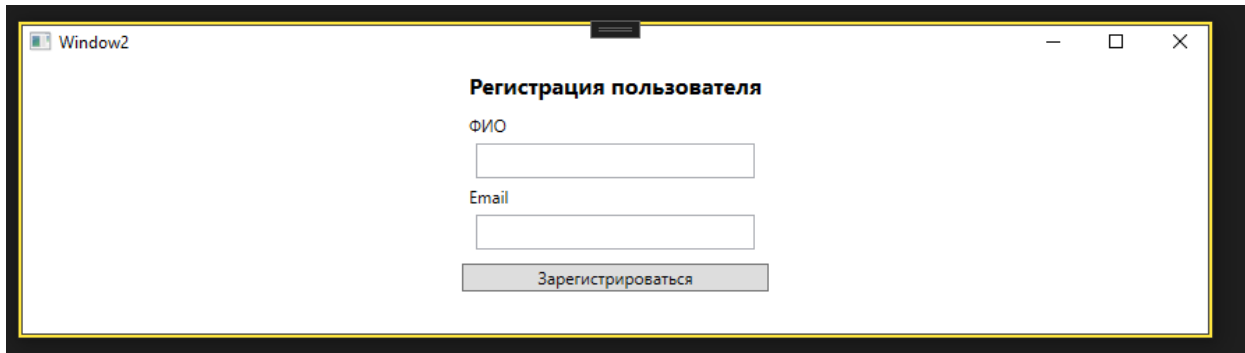
StackPanel располагает содержащиеся в нем элементы управления в виде линейной последовательности: либо в вертикальном столбце, либо в горизонтальной строке.

Характеристики:

- По умолчанию расположение вертикальное (Orientation="Vertical")
- Элементы растягиваются по направлению, перпендикулярному ориентации
- Элементы, выходящие за границы, обрезаются и не отображаются
- Свойства HorizontalAlignment и VerticalAlignment управляют выравниванием

Пример использования StackPanel:

```
9      <Grid>
10      <StackPanel HorizontalAlignment="Center" Orientation="Vertical">
11      <Label Content="Регистрация пользователя" FontSize="16" FontWeight="Bold" />
12      <Label Content="ФИО" />
13      <TextBox Width="200" Height="25" />
14      <Label Content="Email" />
15      <TextBox Width="200" Height="25" />
16      <Button Content="Зарегистрироваться" Margin="0,10,0,0" />
17      </StackPanel>
18    </Grid>
19  </Window>
```



DockPanel – привязка к сторонам

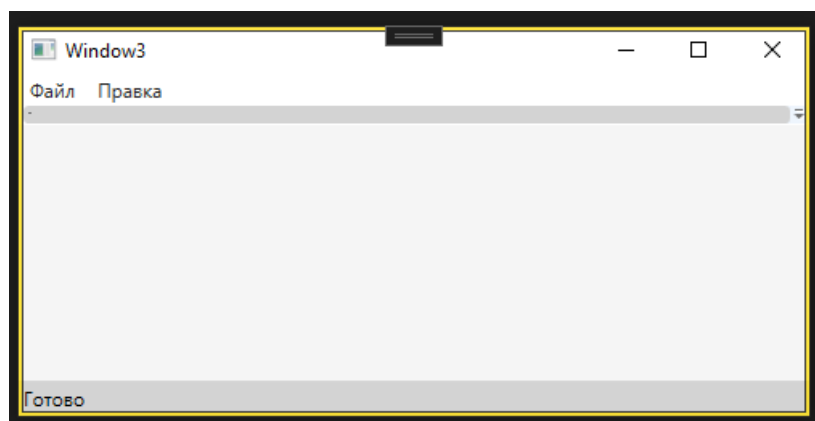
Определение:

DockPanel привязывает содержимое к определенной стороне панели (Top, Bottom, Left, Right).

Характеристики:

- Использует присоединяемое свойство DockPanel.Dock
- Элементы располагаются вдоль границ панели
- Последний элемент заполняет оставшееся пространство (если не установлено LastChildFill="False")

Пример использования DockPanel:



```

8      Title="Window3" Height="200" Width="300">
9      <Grid>
10         <DockPanel LastChildFill="True">
11             <Menu DockPanel.Dock="Top" Background="White">
12                 <MenuItem Header="Файл" />
13                 <MenuItem Header="Правка" />
14             </Menu>
15             <ToolBar DockPanel.Dock="Top" Background="LightGray" />
16             <StatusBar DockPanel.Dock="Bottom" Background="LightGray">
17                 <TextBlock Text="Готово" />
18             </StatusBar>
19             <Grid Background="WhiteSmoke">
20                 <!-- Основное содержимое заполняет оставшееся место -->
21             </Grid>
22         </DockPanel>
23     </Grid>
24 </Window>

```

Grid – табличная сетка

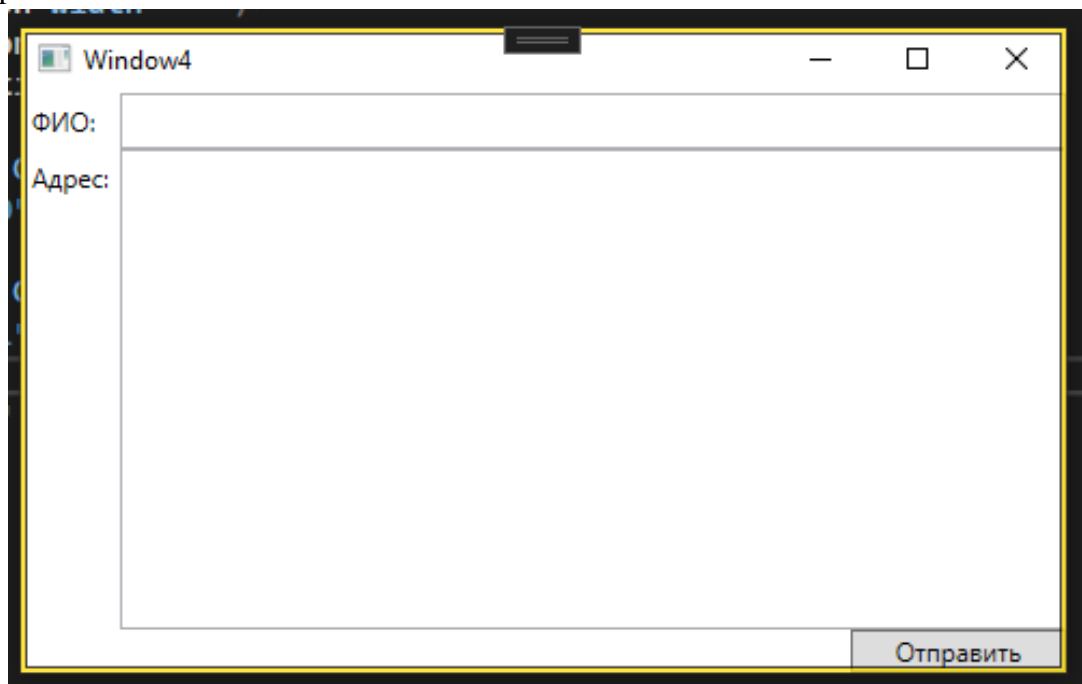
Определение:

Grid располагает содержимое внутри серии ячеек, расположенных в табличной сетке (строки и столбцы).

Характеристики:

- Наиболее универсальная панель размещения
- Определяется количество строк (RowDefinition) и столбцов (ColumnDefinition)
- Использует присоединяемые свойства Grid.Row и Grid.Column
- Поддерживает различные типы размеров: фиксированные (пиксели), автоматические (Auto), пропорциональные (*)

Пример использования Grid:



```

9      <Grid>
10     <Grid.RowDefinitions>
11         <RowDefinition Height="Auto" />
12         <RowDefinition Height="*" />
13         <RowDefinition Height="Auto" />
14     </Grid.RowDefinitions>
15     <Grid.ColumnDefinitions>
16         <ColumnDefinition Width="Auto" />
17         <ColumnDefinition Width="*" />
18         <ColumnDefinition Width="100" />
19     </Grid.ColumnDefinitions>
20
21     <Label Grid.Row="0" Grid.Column="0" Content="ФИО:" />
22     <TextBox Grid.Row="0" Grid.Column="1" Grid.ColumnSpan="2" />
23
24     <Label Grid.Row="1" Grid.Column="0" Content="Адрес:" />
25     <TextBox Grid.Row="1" Grid.Column="1" Grid.ColumnSpan="2" />
26
27     <Button Grid.Row="2" Grid.Column="2" Content="Отправить" />
28 </Grid>

```

Присоединяемые свойства XAML

Присоединяемые свойства позволяют дочернему элементу устанавливать значение свойства, которое в действительности определено в родительском элементе.

Синтаксис:

<РодительскийЭлемент>

 <ДочернийЭлемент РодительскийЭлемент.СвойствоРодителя="Значение" />

</РодительскийЭлемент>

Примеры присоединяемых свойств:

Canvas.Left, Canvas.Top – позиция в Canvas

Grid.Row, Grid.Column – позиция в Grid

DockPanel.Dock – сторона в DockPanel

Пример:

<Canvas>

 <Button Canvas.Left="10" Canvas.Top="20" Content="Кнопка" />

</Canvas>

Свойство Canvas.Left определено в классе Canvas, но используется на элементе Button.

Элементы управления

Кнопка

Button – это элемент управления, представляющий обычную интерактивную кнопку, которая срабатывает при нажатии пользователем.

Основные свойства и события:

Свойство/Событие	Описание
Content	Текст или содержимое кнопки
Click	Событие, возникающее при нажатии на кнопку
IsDefault	Если True, кнопка срабатывает при нажатии Enter
IsCancel	Если True, кнопка срабатывает при нажатии Esc
IsEnabled	Включена ли кнопка

Пример использования Button:

```
9      <Grid>
10     <Button x:Name="BtnOK"
11           Content="OK"
12           Width="100"
13           Height="40"
14           IsDefault="True"
15           Click="BtnOK_Click" />
16     </Grid>

27     private void BtnOK_Click(object sender, RoutedEventArgs e)
28     {
29         MessageBox.Show("Кнопка нажата!");
30     }
31 }
```

Переключаемая кнопка

ToggleButton – это кнопка, которая может находиться в двух состояниях: нажатом (IsChecked=True) и отжатом (IsChecked=False).

Основные свойства и события:

Свойство/Событие	Описание
IsChecked	Состояние кнопки (True/False)
Click	Событие при нажатии или отжатии
Checked	Событие при нажатии кнопки
Unchecked	Событие при отжатии кнопки

Пример использования ToggleButton:

```
9      <Grid>
10     <ToggleButton x:Name="ToggleBtnSound"
11           Content="Звук: ВКЛ"
12           Checked="ToggleBtnSound_Checked"
13           Unchecked="ToggleBtnSound_Unchecked"
14           Width="50"
15           Height="50" />
16     </Grid>

27     private void ToggleBtnSound_Checked(object sender, RoutedEventArgs e)
28     {
29         MessageBox.Show("Звук включен");
30     }
31
32     private void ToggleBtnSound_Unchecked(object sender, RoutedEventArgs e)
33     {
34         MessageBox.Show("Звук выключен");
35     }
```

Независимый переключатель

CheckBox – это элемент управления, позволяющий пользователю выбрать одну или несколько независимых опций. Несколько CheckBox могут быть выбраны одновременно.

Характеристики:

- Наследует от класса ToggleButton

- Использует свойство IsChecked для определения состояния
- Независимы друг от друга (можно выбрать несколько)

Пример использования CheckBox:

```

9      <Grid>
10     <CheckBox x:Name="CheckBoxTerms"
11             Content="Я согласен с условиями использования" />
12     <CheckBox x:Name="CheckBoxNewsletter"
13             Content="Подписаться на новости" Margin="0,0,0,400" VerticalAlignment="Bottom"/>
14     </Grid>

29     if (CheckBoxTerms.IsChecked == true)
30     {
31         MessageBox.Show("Условия приняты");
32     }

```

Зависимый переключатель

RadioButton — это элемент управления для выбора одной опции из нескольких взаимоисключающих вариантов. В одной группе только одна опция может быть выбрана одновременно.

Характеристики:

- Наследует от класса ToggleButton
- Использует свойство GroupName для группировки в одну область выбора
- Только один RadioButton в группе может быть IsChecked=True

Пример использования RadioButton:

```

9      <Grid>
10     <GroupBox Header="Выберите загрузочное устройство:">
11         <StackPanel>
12             <RadioButton GroupName="Boot" x:Name="RbHardDisk"
13                 Content="Жесткий диск" IsChecked="True" Checked="RbHardDisk_Checked" />
14             <RadioButton GroupName="Boot" x:Name="RbDvdDrive"
15                 Content="DVD-привод" Checked="RbDvdDrive_Checked" />
16             <RadioButton GroupName="Boot" x:Name="RbUsb"
17                 Content="USB-флешка" Checked="RbUsb_Checked" />
18         </StackPanel>
19     </GroupBox>
20 </Grid>

20     public partial class Window5 : Window
21     {
22         public Window5()
23         {
24             InitializeComponent();
25         }
26
27         private void RbHardDisk_Checked(object sender, RoutedEventArgs e)
28         {
29             if (RbHardDisk.IsChecked == true)
30             {
31                 MessageBox.Show("Загрузка с жесткого диска");
32             }
33         }
34
35         private void RbDvdDrive_Checked(object sender, RoutedEventArgs e)
36         {
37             if (RbDvdDrive.IsChecked == true)
38             {
39                 MessageBox.Show("Загрузка с DVD-привода");
40             }
41         }
42
43         private void RbUsb_Checked(object sender, RoutedEventArgs e)
44         {
45             if (RbUsb.IsChecked == true)
46             {
47                 MessageBox.Show("Загрузка с USB-флешки");
48             }
49         }
50     }
51 }

```

Выпадающий список

ComboBox – это элемент управления, представляющий выпадающий список, из которого пользователь может выбрать один элемент.

Характеристики:

- Содержит коллекцию элементов `ComboBoxItem`
- Заголовок содержимого может быть текстом или любым другим элементом управления
- Позволяет определить выбранный элемент несколькими способами

Способы определения выбранного элемента:

Свойство	Описание
<code>SelectedIndex</code>	Числовой индекс выбранного элемента (-1 = нет выбора)
<code>SelectedItem</code>	Объект, выбранный в списке
<code>SelectedValue</code>	Значение выбранного объекта

Пример использования `ComboBox`:

```
9      <Grid>
10      <ComboBox x:Name="ComboBoxColor" SelectedIndex="1"
11                Width="100" Height="100">
12          <ComboBoxItem Content="Красный" Selected="ComboBoxItem_Selected"/>
13          <ComboBoxItem Content="Зеленый" />
14          <ComboBoxItem Content="Синий" />
15        </ComboBox>
16      </Grid>
17    </Window>
```

```
28    private void ComboBoxItem_Selected(object sender, RoutedEventArgs e)
29    {
30        string selectedColor = (ComboBoxColor.SelectedItem as ComboBoxItem).Content.ToString();
31        MessageBox.Show("Выбран цвет: " + selectedColor);
32    }
```

Список

ListBox – это элемент управления, представляющий список, из которого пользователь может выбрать один или несколько элементов (в зависимости от настроек).

Характеристики:

- Содержит коллекцию элементов `ListBoxItem`
- Отображает все элементы сразу (в отличие от `ComboBox`)
- Поддерживает выбор одного или нескольких элементов
- Способы определения выбранного элемента аналогичны `ComboBox`

Пример использования `ListBox`:

```
<ListBox x:Name="ListBoxItems" SelectedIndex="0" Height="150">
  <ListBoxItem Content="Элемент 1" />
  <ListBoxItem Content="Элемент 2" />
  <ListBoxItem Content="Элемент 3" />
  <ListBoxItem Content="Элемент 4" />
</ListBox>
```

Ползунок

Slider – это элемент управления, представляющий ползунок для выбора числового значения в диапазоне.

Основные свойства:	
Свойство	Описание
Minimum	Минимальное значение
Maximum	Максимальное значение
Value	Текущее значение
SmallChange	Изменение при нажатии стрелок
LargeChange	Изменение при нажатии на полосу
Orientation	Вертикальное или горизонтальное расположение

Пример использования Slider:

```
<Slider x:Name="SliderVolume"
    Height="25"
    Width="200"
    Minimum="0"
    Maximum="100"
    Value="50"
    SmallChange="1"
    LargeChange="10" />
<TextBlock Text="{Binding ElementName=SliderVolume, Path=Value}" />
```

Меню (Menu)

Menu – это элемент управления для создания иерархического меню. Содержит коллекцию объектов MenuItem, которые могут содержать другие MenuItem и разделители (Separator).

Характеристики:

- Каждый MenuItem может содержать подменю
- Знак подчеркивания (_) в заголовке обозначает горячую клавишу
- Может содержать RadioButton и CheckBox для специальных опций

Пример использования Menu:

```
<Menu Background="White" BorderBrush="Navy" BorderThickness="1">
    <MenuItem Header="_Файл">
        <MenuItem Header="_Открыть" Click="MenuOpen_Click" />
        <MenuItem Header="_Сохранить" Click="MenuSave_Click" />
        <Separator />
        <MenuItem Header="_Выход" Click="MenuExit_Click" />
    </MenuItem>
    <MenuItem Header="_Вид">
        <MenuItem Header="_Полноэкранный" IsCheckable="True" />
    </MenuItem>
    <MenuItem Header="_О программе" Click="MenuAbout_Click" />
</Menu>
```

```
private void MenuOpen_Click(object sender, RoutedEventArgs e)
{
    MessageBox.Show("Открытие файла...");
}
```

Панель инструментов (ToolBar)

ToolBar – это элемент управления для создания панели инструментов с кнопками и другими элементами управления для быстрого доступа к часто используемым командам.

Характеристики:

- Может содержать любые элементы управления (кнопки, картинки, разделители)
- Используется как альтернатива меню
- Несколько ToolBar помещаются в ToolBarTray

Пример использования ToolBar:

```
<ToolBarTray Background="LightGray">
  <ToolBar>
    <Button Click="ButtonOpen_Click">
      <Image Source="open.png" Width="20" Height="20" />
    </Button>
    <Button Click="ButtonSave_Click">
      <Image Source="save.png" Width="20" Height="20" />
    </Button>
    <Separator />
    <Button Click="ButtonHelp_Click">
      <Image Source="help.png" Width="20" Height="20" />
    </Button>
  </ToolBar>
</ToolBarTray>
```

Строка состояния (StatusBar)

StatusBar – это элемент управления, предназначенный для отображения информации о состоянии приложения внизу окна.

Характеристики:

- Может содержать любые элементы управления
- Часто содержит StatusBarItem для структурирования информации
- Использует DockPanel.Dock="Bottom" для крепления снизу окна

Пример использования StatusBar:

```
<StatusBar DockPanel.Dock="Bottom" Background="LightGray">
  <TextBlock Text="Сохранение документа..." />
  <StatusBarItem HorizontalAlignment="Right">
    <TextBlock Text="Подключение к БД: ОК" />
  </StatusBarItem>
</StatusBar>
```

холст для рисования

InkCanvas – это элемент управления, позволяющий пользователю рисовать и редактировать линии с помощью мыши или пера.

Основные свойства:

Свойство	Описание
DefaultDrawingAttributes	Параметры кисти (цвет, ширина, высота)
EditingMode	Режим: Ink (рисование), Select (выбор), EraseByPoint, EraseByStroke
Width, Height	Размеры холста

Пример использования InkCanvas:

```
<InkCanvas x:Name="DrawingCanvas" Height="300" Width="400">
  <InkCanvas.DefaultDrawingAttributes>
    <DrawingAttributes Color="Red" Height="3" Width="3" />
  </InkCanvas.DefaultDrawingAttributes>
</InkCanvas>
```

// Изменение режима редактирования

DrawingCanvas.EditingMode = InkCanvasEditingMode.Ink; // Рисование

DrawingCanvas.EditingMode = InkCanvasEditingMode.EraseByStroke; // Удаление

Обработка событий

Определение события

Событие – это сигнал, который генерируется элементом управления в ответ на действия пользователя (клик, движение мыши, нажатие клавиши и т.д.) или другие системные события.

Синтаксис добавления обработчика события

Способ 1: Прямое указание в XAML

```
<Button Content="Нажми меня" Click="Button_Click" />
```

Способ 2: Использование конструктора событий в Visual Studio

В Visual Studio можно выбрать элемент управления и в панели Properties нажать на иконку события (молния), затем дважды щелкнуть на нужном событии. IDE автоматически создаст метод-обработчик.

Реализация обработчика события в коде

```
private void Button_Click(object sender, RoutedEventArgs e)
{
    MessageBox.Show("Кнопка нажата!");
}
```

Параметры:

sender – объект, который вызвал событие

e – аргументы события (RoutedEventArgs или более специализированный класс)

Доступ к элементам управления

Способ 1: По имени (требуется атрибута x:Name или Name)

```
<Button x:Name="MyButton" Content="Кнопка" Click="MyButton_Click" />
```

```
private void MyButton_Click(object sender, RoutedEventArgs e)
{
    MyButton.Background = Brushes.LightGreen;
}
```

Способ 2: Через параметр sender

```
private void Button_Click(object sender, RoutedEventArgs e)
{
    Button button = sender as Button;
    button.Background = Brushes.Yellow;
}
```

Способ 3: Использование свойства Tag для идентификации

```
<MenuItem Header="_Открыть" Tag="open" Click="MenuItem_Click" />
<MenuItem Header="_Сохранить" Tag="save" Click="MenuItem_Click" />
```

```
private void MenuItem_Click(object sender, RoutedEventArgs e)
{
    string tag = ((FrameworkElement)sender).Tag.ToString();

    if (tag == "open")
        MessageBox.Show("Открытие файла...");
    else if (tag == "save")
        MessageBox.Show("Сохранение файла...");
}
```

Наиболее часто используемые события

Событие	Описание	Применяется к
Click	Нажатие на элемент управления	Button, MenuItem
MouseMove	Движение указателя мыши по элементу	Все элементы
MouseEnter	Вход указателя мыши в границы элемента	Все элементы
MouseLeave	Выход указателя мыши из границ элемента	Все элементы
MouseDown	Нажатие кнопки мыши на элементе	Все элементы
MouseUp	Отпускание кнопки мыши на элементе	Все элементы
MouseWheel	Прокрутка колесика мыши	Все элементы
KeyDown	Нажатие клавиши (элемент имеет фокус)	Все элементы
KeyUp	Отпускание клавиши (элемент имеет фокус)	Все элементы
Checked	Переключатель нажат (ToggleButton, CheckBox, RadioButton)	Переключатели
Unchecked	Переключатель отжат	Переключатели